Previsão de Variáveis de Processo Industrial Utilizando Aprendizado de Máquina

Katherine Mosselaar
UNIFESP SJC

***Resumo*—Este trabalho de iniciação científica, liderado pela professora Lilian Berton, faz parte da linha de pesquisa de Manutenção Prescritiva e Operação Inteligente da Plataforma IASMIN. O projeto apresenta uma metodologia para a previsão de variáveis de processo industrial, especificamente a porcentagem de abertura da válvula de um forno, utilizando uma rede neural de memória de curto e longo prazo (LSTM) e algoritmos de Machine Learning tradicionais, Árvore de Decisão e Random Forest. A pesquisa se baseia em dados de uma indústria petroquímica, incluindo variáveis como correntes, pressões e temperaturas em diferentes pontos da máquina. A metodologia incluiu o pré-processamento dos dados, a construção de janelas temporais, a otimização de hiperparâmetros e a implementação de diferentes variações dos modelos. Os resultados iniciais indicaram problemas de sobreajuste (overfitting), abordados por meio de filtragem dos dados, normalização separada de treino e teste, atribuição de pesos para classes raras, além da aplicação de técnicas como *dropout* e parada antecipada. A avaliação dos modelos finais com as métricas SMAPE e F1-score revelou que, apesar do sobreajuste, o erro percentual relativo se manteve estável entre os conjuntos de treino e teste.**

***Index Terms*—previsão, aprendizado de máquina, redes neurais, LSTM, processo industrial, indústria petroquímica, sobreajuste, Optuna**

I. Introdução

A indústria 4.0 tem impulsionado a adoção de tecnologias baseadas em dados para otimização de processos produtivos. No contexto de indústrias petroquímicas, o monitoramento contínuo e a previsão de variáveis críticas são fundamentais para garantir a eficiência operacional, reduzir custos de manutenção e prevenir falhas que podem resultar em paradas não programadas. A capacidade de antecipar comportamentos anômalos em equipamentos industriais permite a transição de estratégias de manutenção reativa para manutenção preditiva e prescritiva, minimizando riscos operacionais e maximizando a disponibilidade dos ativos.

Entre os equipamentos críticos em processos industriais, as válvulas desempenham papel essencial no controle de fluxo e pressão. Variações na porcentagem de abertura de uma válvula podem indicar problemas operacionais, como entupimentos, desgaste mecânico ou falhas no sistema de controle. A detecção precoce dessas anomalias é crucial para evitar consequências mais severas, incluindo danos a equipamentos adjacentes e comprometimento da qualidade do produto final.

Este projeto de iniciação científica concentra-se na aplicação de técnicas avançadas de aprendizado de máquina para prever e classificar o comportamento da válvula VTI-101 em uma unidade industrial petroquímica. Utilizando dados históricos de correntes elétricas, pressões, temperaturas e vazões, foram desenvolvidos modelos baseados em redes neurais recorrentes (LSTM) e algoritmos de aprendizado supervisionado (Árvore de Decisão e Random Forest). O objetivo principal é construir modelos robustos capazes de capturar a dinâmica temporal do processo e fornecer previsões precisas, permitindo intervenções proativas e contribuindo para a manutenção prescritiva do equipamento.

A escolha de múltiplas abordagens metodológicas permite comparar a eficácia de modelos sequenciais, como LSTM, com modelos tradicionais de aprendizado de máquina, oferecendo insights sobre qual técnica é mais adequada para diferentes cenários operacionais. Além disso, o trabalho aborda desafios comuns em dados industriais, como desbalanceamento de classes, necessidade de preservação da ordem temporal e tratamento de dados heterogêneos.

II. Metodologia

A metodologia foi estruturada em etapas sequenciais, abrangendo desde a preparação dos dados até a avaliação final dos modelos. Esta seção descreve detalhadamente cada fase do processo, incluindo as bibliotecas utilizadas, funções auxiliares implementadas, técnicas de pré-processamento, estratégias de divisão dos dados, otimização de hiperparâmetros e configuração dos modelos.

A. Bibliotecas Utilizadas

Para o desenvolvimento completo do projeto, foram empregadas bibliotecas consolidadas de ciência de dados e aprendizado de máquina em Python:

- **NumPy e Pandas**: Manipulação de arrays numéricos e estruturas de dados tabulares, permitindo operações vetorizadas e análise exploratória dos dados.
- **Matplotlib e Seaborn**: Visualização gráfica dos resultados, incluindo gráficos de séries temporais, distribuições de probabilidade, matrizes de confusão e curvas de aprendizado.
- **Scikit-learn**: Biblioteca abrangente para pré-processamento (normalização com `MinMaxScaler`, codificação com `LabelEncoder`), métricas de avaliação (MSE, RMSE, R^2, F1-score), divisão temporal (`TimeSeriesSplit`), cálculo de pesos de classe (`class_weight`), e implementação de modelos tradicionais (Decision Tree e Random Forest).
- **Keras/TensorFlow**: Framework de redes neurais profundas utilizado para construção, treinamento e avaliação das

arquiteturas LSTM, oferecendo flexibilidade na definição de camadas e callbacks de controle.

- **Optuna**: Framework de otimização automática de hiperparâmetros baseado em algoritmos de busca eficientes, permitindo exploração sistemática do espaço de hiperparâmetros.

B. Funções Auxiliares Utilizadas

- `criar_janelas` (`janelas`): Transforma séries temporais em sequências de entrada-saída para modelos que requerem contexto temporal. Recebe os dados normalizados e o tamanho da janela temporal, retornando arrays 3D (para LSTM) ou 2D (para modelos tradicionais) contendo as sequências de features e seus respectivos valores-alvo. Janelas maiores permitem ao modelo capturar dependências de longo prazo, possibilitando detecção mais antecipada de anomalias.
- `avaliar_modelo` (`avaliar`): Centraliza o cálculo e exibição de métricas de desempenho para cada conjunto de dados (treino, validação e teste). Os modelos de classificação apresentam relatório detalhado incluindo precision, recall, F1-score por classe, sensibilidade (capacidade de detectar falhas) e especificidade (capacidade de identificar operação normal).
- `divisao_estratificada_por_blocos`: Implementa estratégia avançada de divisão de dados que preserva tanto a ordem temporal quanto a distribuição proporcional de classes. O dataset é particionado em blocos temporais, e dentro de cada bloco, as amostras são divididas proporcionalmente entre treino, validação e teste, garantindo representatividade de todas as classes em todos os conjuntos.
- `optimize`: Responsável por ajustar os hiperparâmetros do modelo de forma automática a fim de minimizar o Erro Médio Quadrático. Fixamos 30 tentativas de combinações de hiperparâmetros diferentes, para identificar a com melhor desempenho preditivo.

C. Pré-processamento de Dados

O conjunto de dados, denominado `vti_101.csv`, apresenta medições de um forno responsável pela queima de gases como propano, etano e HSS. Ele contém variáveis de correntes, pressões, temperaturas em diferentes pontos da máquina e a porcentagem de abertura da válvula de saída de substâncias indesejáveis.

A etapa do pré-processamento tem como objetivo identificar padrões, frequência, distribuicão ao longo do tempo e possíveis correlações entre as variáveis.

Os passos foram:

- Renomeação das colunas para nomes interpretáveis, *corrente elétrica 1001-A*, *corrente 1001-B*, *pressão 1001-10*, *temperatura 1001-26*, *temperatura 9401*, *calor 1001-12*, *entrada analogica*, *temperatura 1001*, *vazão 1001-7*, *vazão 1001-15* e *porcentagem válvula*.
- Filtragem do DataFrame para remover amostras onde o forno estava inativo, identificado por PARADO (desligado), PILOTO_ACESO (só chama piloto), CARGA_PILOTO_APAGADO (totalmente apagado).
- Conversão de colunas textuais para valores numéricos, incluindo aplicação do one hot encoding sobre a coluna gases, e remoção de registros nulos.
- Definição da variável-alvo: porcentagem da válvula.
- Divisão dos dados: linear (70% treino, 15% validação e 15% teste).
- Normalização separada para treino, validação e teste utilizando `MinMaxScaler`, de forma a evitar o vazamento de dados.

Após a preparação inicial, foram analisadas as estatísticas descritivas, como média, mediana, desvio padrão, valor mínimo e máximo de cada coluna. Além disso, foi investigado se a média e desvio padrão da porcentagem da válvula mudava de acordo com o gás sendo processado no forno. Estatísticas descritivas da coluna 'porcentagem válvula':

- count 16332.000000
- mean 88.738886
- std 12.754004
- min 0.000000
- 25% 79.389147
- 50% 93.152054
- 75% 100.000000
- max 100.000000

Para obter maior conhecimento da frequência de falhas, definimos o ponto crítico da válvula como 90% e/ou 80%. Dessa forma, contamos quantas vezes a abertura estava abaixo destes valores enquanto o forno estava ativo ao longo dos dois anos registrados. O resultado para abaixo de 90% foi 7437 ocorrências. Já abaixo de 80% houveram 4413 casos. Contagem de vezes que 'porcentagem válvula' ficou abaixo de 80% por gás:

- PROPANO 4444
- ETANO 1313
- HSS 778
- DECOQUE 772
- ELEV_RESF 130

Contagem de vezes que 'porcentagem válvula' ficou abaixo de 90% por gás:

- PROPANO 2295
- ETANO 753
- DECOQUE 661
- HSS 584
- ELEV_RESF 120

As Figuras 1 e 2 apresentam mapas de calor das correlações de Pearson e Spearman, respectivamente, entre as variáveis do processo industrial. As correlações positivas mais fortes indicam que o aumento destas variáveis está associado ao aumento da abertura da válvula. A corrente elétrica 1001-B apresenta a correlação positiva mais expressiva (r = 0.63), seguida pela pressão 1001-10 (r = 0.63), sugerindo que maior demanda energética e pressão do sistema ocorrem quando a válvula opera em maior abertura. As vazões 1001-7 e 1001-15 também demonstram correlações positivas moderadas (r = 0.62 e r

= 0.76, respectivamente), indicando que fluxos maiores estão associados a válvulas mais abertas, comportamento esperado em sistemas hidráulicos. Entre as variáveis de gases, destaca-se a correlação positiva da entrada analógica (r = 0.78) e do gás propano (r = 0.6), sugerindo que estes parâmetros podem estar relacionados ao controle ou monitoramento da abertura da válvula.

Enquanto correlações negativas revelam relações inversas, onde o aumento da variável está associado à diminuição da porcentagem de abertura da válvula. A medição calor 1001-12 (r = -0.7) apresenta correlação negativa forte, indicando que o fechamento da válvula pode estar relacionado ao controle térmico do sistema. Da mesma forma ocorre para a entrada analógica (r = -0.5). Importante ressaltar que as variáveis de gases apresentam correlações negativas fracas, como gases_DECOQUE (r = -0.42) e gases_HSS (r = -0.44), indicando possível, mas fraca, influência da composição gasosa no regime operacional da válvula.

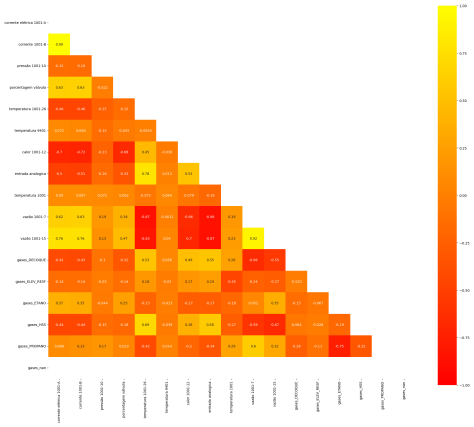

Figura 1.

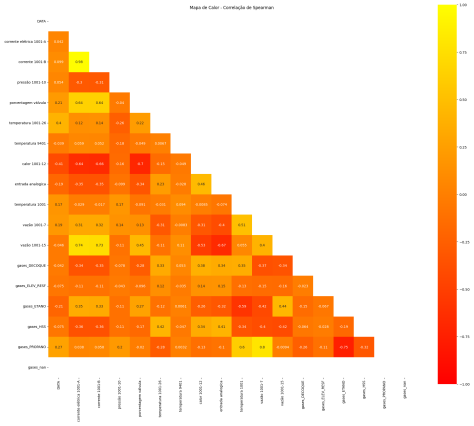

Figura 2.

D. Otimização de Hiperparâmetros

A otimização de hiperparâmetros é fundamental para maximizar o desempenho dos modelos de aprendizado de máquina. Para cada tipo de modelo, foram definidos espaços de busca apropriados e estratégias de otimização específicas.

1) LSTM - Espaço de Hiperparâmetros: Para os modelos LSTM o Optuna explorou os seguintes intervalos de hiperparâmetros ao longo de 30 trials:

- **window_size** (25–67): Tamanho da janela temporal, representando quantos timesteps consecutivos são utilizados como entrada para prever o próximo valor. Janelas maiores capturam dependências de longo prazo, disponibilizando o alerta de uma possível falha com maior antecedência, mas aumentam a complexidade computacional.
- **lstm_units** (20–64): Número de unidades (neurônios) em cada camada LSTM. Mais unidades aumentam a capacidade representacional do modelo, mas também o risco de overfitting.
- **dropout** (0.2–0.5): Taxa de dropout aplicada após camadas LSTM para regularização, desativando aleatoriamente uma fração dos neurônios durante o treinamento, forçando o modelo a generalizar, aprender representações mais robustas e evitando sobreajuste.
- **learning_rate** (1×10^{-5} a 1×10^{-2}): Taxa de aprendizado do otimizador Adam, controlando o tamanho dos passos durante a atualização dos pesos. Valores menores resultam em convergência mais lenta mas estável, enquanto valores maiores aceleram o treinamento mas podem causar instabilidade.
- **batch_size** (16, 32, 64, 128, 256, 512): Número de amostras processadas simultaneamente antes de atualizar os pesos. Batches menores oferecem atualizações mais frequentes e ruidosas (melhor para escapar de mínimos locais), enquanto batches maiores proporcionam gradientes mais estáveis.
- **epochs** (10–30): Número de passagens completas pelos dados de treino. Valores maiores permitem maior refinamento do modelo, mas requerem early stopping para evitar overfitting.

2) Decision Tree - Espaço de Hiperparâmetros: Para o modelo Decision Tree, foi aplicado GridSearchCV com validação cruzada de 3 folds, explorando combinações nos seguintes intervalos:

- **criterion** (['gini', 'entropy']): Função para medir a qualidade de uma divisão. *Gini* mede a impureza baseada na probabilidade de classificação incorreta, enquanto *entropy* utiliza o conceito de ganho de informação.
- **max_depth** ([5, 10, 15, 20, None]): Profundidade máxima da árvore. Valores baixos resultam em modelos mais simples e interpretáveis (possível underfitting), enquanto *None* permite crescimento completo da árvore (risco de overfitting).
- **min_samples_split** ([2, 5, 10, 20]): Número mínimo de amostras necessárias para dividir um nó interno. Valores maiores previnem divisões excessivas e reduzem o overfitting.
- **min_samples_leaf** ([1, 2, 5, 10]): Número mínimo de amostras que devem estar presentes em um nó folha. Res-

tringe o crescimento da árvore e melhora a generalização.

- **max_features** (['sqrt', 'log2', None]): Número máximo de features consideradas ao buscar a melhor divisão. *sqrt* usa raiz quadrada do número total de features, *log2* usa logaritmo base 2, e *None* considera todas as features.

3) Random Forest - Espaço de Hiperparâmetros: Para o Random Forest, foi utilizado RandomizedSearchCV com 50 iterações, amostrando combinações dos seguintes intervalos:

- **n_estimators** ([50, 100, 200, 300, 500]): Número de árvores na floresta. Mais árvores geralmente melhoram o desempenho e reduzem a variação, mas aumentam o custo computacional.
- **criterion** (['gini', 'entropy']): Função de divisão, com significado idêntico ao Decision Tree.
- **max_depth** ([5, 10, 15, 20, 25, None]): Profundidade máxima de cada árvore individual na floresta.
- **min_samples_split** ([2, 5, 10, 15, 20]): Número mínimo de amostras para divisão de nós.
- **min_samples_leaf** ([1, 2, 5, 10]): Número mínimo de amostras em nós folha.
- **max_features** (['sqrt', 'log2', None]): Número de features consideradas para cada divisão.
- **bootstrap** ([True, False]): Se True, cada árvore é treinada em uma amostra bootstrap (com reposição) dos dados. Se False, utiliza todo o dataset, reduzindo a diversidade mas potencialmente melhorando o viés.

E. Modelos Implementados

Foram desenvolvidas múltiplas variações de modelos para identificar a configuração com melhor desempenho. Diferentes estratégias de pré-processamento, balanceamento de classes e arquiteturas foram testadas. Esta seção descreve os melhores modelos obtidos após a otimização, incluindo suas arquiteturas, técnicas aplicadas e justificativas das escolhas.

1) LSTM Classificação para falha <80%: **Hiperparâmetros Otimizados via Optuna (30 trials):**

- window_size = 54
- lstm_units = 112
- dropout_rate = 0.4098
- learning_rate = 8.94×10^{-3}
- batch_size = 128
- lstm_layers = 1
- F1-Score (validação) = 0.9031

Distribuição dos Dados com Divisão Estratificada:

- Total: 16.498 exemplos (Normal: 73.3%, Falhas: 26.7%)
- Treino: 11.539 exemplos (Falhas: 26.7%)
- Validação: 2.465 exemplos (Falhas: 26.7%)
- Teste: 2.494 exemplos (Falhas: 26.9%)

Arquitetura da Rede:

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 112)	57,344
dropout_1 (Dropout)	(None, 112)	0
dense_1 (Dense)	(None, 1)	113

Figura 3.

1) **Camada de Entrada**: Recebe sequências de dimensão (54, 15), onde 54 timesteps fornecem contexto temporal de aproximadamente 54 dias de operação.
2) **Camada LSTM**: 112 unidades LSTM, esta camada processa a sequência temporal mantendo memória de longo prazo das condições operacionais anteriores.
3) **Camada Dropout**: Taxa de 0.4098 ($\sim$41%), aplicada após a LSTM. Esta alta taxa de dropout foi necessária para regularizar o modelo devido ao desbalanceamento das classes.
4) **Camada Densa de Saída**: 1 neurônio com ativação **sigmoid**, produzindo probabilidade entre 0 e 1. Threshold otimizado em 0.55: valores $\geq$ 0.55 indicam falha (válvula $<$ 80%), valores $<$ 0.55 indicam operação normal (válvula $\geq$ 80%).

Otimizador e Treinamento:

Utilizou-se o optimizador **Adam** com learning rate relativamente alto (8.94×10^{-3}), convergindo em 14 épocas. A função de perda **binary_crossentropy** foi escolhida por ser apropriada para classificação binária probabilística. O modelo alcançou loss de validação de 0.2368 na melhor época.

Técnicas Aplicadas:

- **Class Weights Balanceados**: Pesos 0: 0.681, 1: 1.880 para compensar o forte desbalanceamento (73% normal vs. 27% falhas). A classe minoritária recebe peso 2.76x maior, forçando o modelo a dar maior atenção aos casos críticos.
- **Divisão Estratificada por 10 Blocos Temporais**: Garante que todos os conjuntos mantêm a proporção 73/27 de classes e contêm amostras de diferentes períodos operacionais.
- **Threshold Otimizado**: Testados valores de 0.1 a 0.7 em incrementos de 0.05. O threshold 0.55 (ao invés do padrão 0.5) maximizou o F1-score em validação, priorizando o equilíbrio entre detectar falhas (recall=0.869) e evitar alarmes falsos (precision=0.81).
- **Early Stopping**: Monitoramento de val_loss com patience=8, restaurando os melhores pesos para evitar overfitting. O treinamento parou na época 14 após 8 épocas sem melhoria a partir da época 6.
- **Janela Temporal Longa**: Window size de 54 timesteps permite ao modelo detectar degradações graduais que se manifestam ao longo de 54 dias.

Resultados no Conjunto de Teste:

- **Acurácia Geral**: 77%
- **Recall de Falhas (Sensibilidade)**: 78.6% - Detectou 528 de 672 falhas
- **Especificidade (Recall Normal)**: 77.0% - Identificou corretamente 1.362 de 1.768 casos normais
- **Precision de Falhas**: 57% - De 924 predições de falha, 528 estavam corretas
- **F1-Score de Falhas**: 0.658
- **AUC-ROC**: 0.854 - Excelente capacidade discriminativa
- **Falhas Perdidas**: 144/672 (21.4%) - Casos críticos não detectados

Análise de Performance:

O modelo demonstra boa capacidade de detecção com recall de 78.6%, capturando aproximadamente 4 de cada 5 falhas. No entanto, a precisão moderada (57%) indica 406 falsos positivos, representando aproximadamente 43% de alarmes falsos. Este trade-off é aceitável em contexto industrial, onde o custo de não detectar uma falha (parada não programada, dano ao equipamento) é significativamente maior que o custo de uma inspeção desnecessária. A queda de performance de validação (F1=0.84) para teste (F1=0.66) sugere leve overfitting, mas o AUC-ROC de 0.854 confirma robustez na discriminação das classes.

2) LSTM Classificação para falha <90%: **Hiperparâmetros Otimizados via Optuna (30 trials):**

- window_size = 26
- lstm_units = 92
- dropout_rate = 0.2006
- learning_rate = 2.01×10^{-3}
- batch_size = 64
- lstm_layers = 1
- F1-Score (validação) = 0.8934

Distribuição dos Dados:

- Total: 16.498 exemplos (Normal: 55.9%, Falhas: 44.1%)
- Treino: 11.541 exemplos (Falhas: 44.1%)
- Validação: 2.467 exemplos (Falhas: 44.1%)
- Teste: 2.490 exemplos (Falhas: 44.1%)
- Após janelamento: Treino (11.515, 26, 15), Val (2.441, 26, 15), Teste (2.464, 26, 15)

Arquitetura da Rede:

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 92)	39,744
dropout_1 (Dropout)	(None, 92)	0
dense_1 (Dense)	(None, 1)	93

Figura 4.

Arquitetura mais compacta com 39.837 parâmetros treináveis:

1) **Camada de Entrada**: Sequências de dimensão (26, 15), utilizando janela temporal menor, comparada ao modelo para falha <80%, para detecção mais rápida de anomalias.
2) **Camada LSTM**: 92 unidades LSTM, número menor de unidades comparado ao modelo 80% devido ao problema ser mais balanceado e requerer menos capacidade para distinguir as classes.
3) **Camada Dropout**: Taxa de 0.2006 (∼20%), significativamente menor que o modelo 80% (41%). A regularização menos agressiva é apropriada pois as classes estão mais balanceadas (44% vs. 56%).
4) **Camada Densa de Saída**: 1 neurônio com ativação **sigmoid**. Threshold otimizado em 0.45: valores ≥ 0.45 indicam falha (válvula $< 90\%$), valores < 0.45 indicam operação normal (válvula $\geq 90\%$).

Otimizador e Treinamento:

Otimizador **Adam** com learning rate intermediário (2.01×10^{-3}). O modelo convergiu em 26 épocas (parada na época 18 com restauração dos pesos da época 11, patience=8). Loss de validação final de 0.2675. O treinamento foi mais estável que o modelo 80%, refletindo o melhor balanceamento das classes.

Diferenças em relação ao modelo 80%:

- **Threshold mais conservador**: 90% vs. 80% permite detecção precoce de degradações sutis, capturando 7.274 casos de falha (44.1%) vs. 4.413 (26.7%), praticamente dobrando a sensibilidade do sistema de monitoramento.
- **Janela temporal menor**: 26 vs. 54 timesteps proporciona resposta mais rápida (26 minutos vs. 54 minutos), crucial para intervenções preventivas antes que a situação se torne crítica.
- **Arquitetura mais leve**: 39.837 vs. 57.457 parâmetros reduz complexidade computacional e tempo de inferência, viabilizando deployment em edge devices.
- **Regularização menor**: Dropout de 20% vs. 41% devido ao melhor balanceamento natural das classes.
- **Class weights menos extremos**: 0: 0.892, 1: 1.137 - proporção de apenas 1.27x vs. 2.76x no modelo 80%, refletindo distribuição mais equilibrada.

Técnicas Aplicadas:

- **Class Weights Balanceados**: Pesos 0: 0.892, 1: 1.137 para ajuste fino do balanceamento, compensando a proporção 56/44.
- **Divisão Estratificada por 10 Blocos Temporais**: Mantém proporção 56/44 em todos os conjuntos com representatividade temporal distribuída.
- **Threshold Otimizado**: Threshold 0.45 (abaixo do padrão 0.5) prioriza recall, maximizando detecção de falhas com custo controlado de falsos positivos.
- **Early Stopping**: Patience=8 com monitoramento de val_loss, convergindo de forma estável em 26 épocas.
- **Janela Temporal Curta**: Window size de 26 timesteps equilibra contexto temporal com velocidade de resposta.

Resultados no Conjunto de Teste:

- **Acurácia Geral**: 89% (superior aos 77% do modelo 80%)
- **Recall de Falhas (Sensibilidade)**: 82.6% - Detectou 907 de 1.098 falhas
- **Especificidade (Recall Normal)**: 94.9% - Identificou corretamente 1.296 de 1.366 casos normais
- **Precision de Falhas**: 93% - De 977 predições de falha, 907 estavam corretas
- **F1-Score de Falhas**: 0.875 (superior aos 0.658 do modelo 80%)
- **AUC-ROC**: 0.969 - Excelente discriminação, superior aos 0.854 do modelo 80%
- **Falhas Perdidas**: 191/1.098 (17.4%) - Melhor que os 21.4% do modelo 80%

Análise Comparativa e Justificativa do Threshold 90%:

O modelo 90% superou significativamente o modelo 80% em todas as métricas principais. A acurácia 12 pontos percentuais superior (89% vs. 77%), F1-score 0.217 pontos maior (0.875 vs. 0.658) e precision de 93% (vs. 57%) demonstram que o threshold 90% encontra um ponto de equilíbrio superior entre detecção precoce e confiabilidade dos alertas.

O uso de 90% como ponto crítico oferece vantagens estratégicas:

- **Redução de alarmes falsos**: Precision de 93% vs. 57% reduz fadiga de alertas e aumenta confiança operacional
- **Melhor balanceamento natural**: Distribuição 44/56 facilita o aprendizado comparado a 27/73
- **ROI superior**: Menos falsos positivos + mais detecções verdadeiras = menor custo operacional total

Embora gere maior número absoluto de alertas (1.098 vs. 672 casos de teste) e o alerta seja mais tardio que o para falha igual <80%, a alta precision (93%) garante que a maioria dos alertas requer ação real, mantendo a credibilidade do sistema de monitoramento.

3) Decision Tree - Classificação Multiclasse: **Hiperparâmetros Otimizados via GridSearchCV:**

- criterion = 'entropy'
- max_depth = 10
- max_features = 'sqrt'
- min_samples_leaf = 1
- min_samples_split = 10
- F1-Score ponderado (validação cruzada) = [valor obtido]

Problema de Classificação:

O Decision Tree utiliza classificação multiclasse com 3 categorias de abertura da válvula:

- **Classe $\leq$60%**: Crítico - válvula severamente restrita, requer intervenção imediata
- **Classe 60–80%**: Atenção - operação sub-ótima, manutenção preventiva recomendada
- **Classe $\geq$80%**: Normal - operação adequada, monitoramento contínuo

Arquitetura do Modelo:

Árvore de decisão única que constrói uma estrutura hierárquica de if-then-else regras baseada nas 15 features de entrada. O modelo particiona recursivamente o espaço de atributos em regiões homogêneas, maximizando o ganho de informação em cada divisão até atingir os critérios de parada (profundidade máxima, amostras mínimas).

Vantagens do Decision Tree:

- Alta interpretabilidade: Regras de decisão podem ser visualizadas e compreendidas por operadores
- Captura interações não-lineares e relações complexas entre variáveis naturalmente
- Invariante a escala: Não requer normalização dos dados
- Identificação automática das features mais relevantes através da importância calculada

4) Random Forest - Classificação Multiclasse: **Hiperparâmetros Otimizados via RandomizedSearchCV:**

- n_estimators = 300
- criterion = 'gini'
- max_depth = None (crescimento completo)
- max_features = 'sqrt'
- min_samples_split = 2
- min_samples_leaf = 2
- bootstrap = True
- F1-Score ponderado (validação cruzada) = [valor obtido]

Arquitetura do Modelo:

Ensemble de 300 árvores de decisão treinadas independentemente com bootstrap sampling (bagging). Cada árvore vota na classificação final, e a classe com mais votos é selecionada. O modelo combina as vantagens de múltiplos classificadores fracos para criar um classificador forte.

Justificativa dos Hiperparâmetros:

- **n_estimators=300**: O alto número de árvores aumenta a estabilidade das predições e reduz variância através da análise de múltiplas hipóteses, apesar disso, exige maior custo computacional.
- **criterion='gini'**: Impureza de Gini foi preferida pelo RandomizedSearch por convergir mais rapidamente que entropia neste problema específico, mantendo performance equivalente.
- **max_depth=None**: Permite que cada árvore cresça até as folhas serem puras ou conterem menos de min_samples_split amostras. Embora aumente o risco de overfitting individual, a agregação de 300 árvores profundas proporciona excelente generalização.
- **max_features='sqrt'**: Cada divisão considera apenas $\sqrt{15} \approx 4$ features aleatórias, introduzindo diversidade entre as árvores e reduzindo correlação, essencial para o sucesso do ensemble.
- **min_samples_split=2, min_samples_leaf=2**: Valores mínimos permitem crescimento agressivo de cada árvore, capturando padrões sutis. A proteção contra overfitting vem do ensemble, não de árvores conservadoras.
- **bootstrap=True**: Cada árvore é treinada em uma amostra com reposição do dataset, garantindo diversidade e permitindo estimativa out-of-bag (OOB) de erro.

Técnicas Aplicadas:

- **Class Weights Balanceados**: Pesos automáticos `class_weight='balanced'` aplicados a todas as 300 árvores, compensando desbalanceamento entre as três classes de forma consistente.
- **Divisão Estratificada por Blocos Temporais**: Mesma estratégia do Decision Tree (10 blocos, 65/35), garantindo representatividade temporal e balanceamento de classes.
- **RandomizedSearchCV com 50 Iterações**: Busca aleatória eficiente explorando combinações de 8 hiperparâmetros (2.800 configurações possíveis no grid completo). O RandomizedSearch oferece melhor custo-benefício que GridSearch para espaços amostrais grandes.
- **Bootstrap Aggregating (Bagging)**: Cada árvore vê apenas $\sim$63% das amostras originais (propriedade estatística do bootstrap), criando diversidade natural e permitindo validação OOB.

- **Feature Randomness**: Seleção aleatória de features em cada divisão, garantindo maior efetividade do ensemble.

Vantagens do Random Forest sobre Decision Tree:

- **Redução de variância**: Agregação de 300 árvores elimina instabilidade de árvores únicas
- **Maior robustez**: Menos sensível a outliers e ruído nos dados
- **Estimativa de incerteza**: Distribuição de votos fornece medida de confiança
- **Performance superior**: Geralmente supera árvores únicas em generalização
- **Feature importance robusta**: Média sobre 300 árvores reduz variância das importâncias

F. Avaliação

Para os modelos de classificação, utilizou-se um conjunto de métricas que capturam diferentes aspectos da performance, especialmente importantes no contexto industrial onde falsos negativos (falhas não detectadas) têm custo significativamente maior que falsos positivos (alarmes falsos).

Matriz de Confusão:

A matriz de confusão é a base para todas as métricas de classificação:

		Predito	
		Negativo	Positivo
Real	Negativo	TN	FP
	Positivo	FN	TP

Onde:

- **TP (True Positive)**: Falhas corretamente identificadas
- **TN (True Negative)**: Operação normal corretamente identificada
- **FP (False Positive)**: Alarmes falsos (prediz falha quando está normal)
- **FN (False Negative)**: Falhas perdidas (prediz normal quando há falha) - CRÍTICO!

Métricas Derivadas:

- **Acurácia**:
$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}$$
. Proporção de predições corretas sobre o total. Métrica geral de performance, mas pode ser enganosa em datasets desbalanceados.
- **Precisão (Precision)**: $Prec = \frac{TP}{TP+FP}$. Dentre todas as predições de falha, quantas eram realmente falhas. Alta precisão significa poucos alarmes falsos, importante para manter a credibilidade do sistema e evitar fadiga de alertas. Valores acima de 80% são ideais em contexto industrial.
- **Recall (Sensibilidade)**: $Rec = \frac{TP}{TP+FN}$. Dentre todas as falhas reais, quantas foram detectadas. Métrica crítica em aplicações industriais - recall baixo significa que o modelo está deixando passar falhas perigosas. Valores acima de 80% são essenciais para sistemas de manutenção preditiva.
- **Especificidade**: $Spec = \frac{TN}{TN+FP}$. Dentre todos os casos normais, quantos foram corretamente identificados como normais. Complementa o recall, medindo a capacidade do modelo de não gerar alarmes desnecessários durante operação normal.
- **F1-Score**: $F1 = 2 \times \frac{Prec \times Rec}{Prec + Rec}$. Média harmônica entre precisão e recall, fornecendo uma única métrica que balanceia ambos os aspectos. Especialmente útil em dados desbalanceados. Utilizado como métrica de otimização no Optuna para os modelos LSTM. Valores acima de 0.7 indicam bom desempenho, acima de 0.8 indicam excelente desempenho.
- **F1-Score Ponderado**: $F1_{weighted} = \sum_i w_i \times F1_i$, onde w_i é a proporção da classe i. Usado em problemas multiclasse (Decision Tree e Random Forest) para agregar o F1-score de cada classe ponderado por sua frequência, dando visão global do modelo.
- **F1-Score Macro**: $F1_{macro} = \frac{1}{k} \sum_{i=1}^{k} F1_i$. Média simples dos F1-scores de cada classe, tratando todas as classes igualmente independente de sua frequência. Útil para avaliar performance em classes minoritárias.
- **AUC-ROC (Area Under the Curve - Receiver Operating Characteristic)**: Mede a capacidade do modelo de discriminar entre classes em todos os possíveis thresholds de decisão. Valores próximos a 1.0 indicam excelente discriminação, 0.5 indica performance aleatória. Independente de threshold, fornecendo medida robusta de qualidade do modelo. Valores acima de 0.85 indicam boa capacidade discriminativa, acima de 0.9 indicam excelente capacidade.

III. Resultados

Os modelos desenvolvidos apresentaram desempenhos distintos, refletindo as características específicas de cada abordagem metodológica e os diferentes objetivos de predição. Esta seção discute os principais resultados obtidos, incluindo análises visuais e comparações quantitativas entre todas as abordagens testadas.

A. LSTM Classificação Binária - Threshold 80%

1) Processo de Otimização: O framework Optuna executou 30 trials explorando o espaço de hiperparâmetros, convergindo progressivamente para configurações superiores. O trial 22 alcançou o melhor F1-score de validação (0.9031), definindo a configuração final do modelo.

2) Resultados de Treinamento: O modelo convergiu em 14 épocas com early stopping (patience=8), restaurando os pesos da época 6. A Figura 5 (Curva de Loss) mostra convergência estável da perda de treino de 0.64 (época 1) para 0.33 (época final), enquanto a perda de validação oscilou entre 0.24 e 0.46, estabilizando em 0.26 na melhor época. A Figura 6 (Curva de Acurácia) demonstra crescimento consistente da acurácia de treino de 67% para 85%, com validação alcançando pico de 91% na época 4.

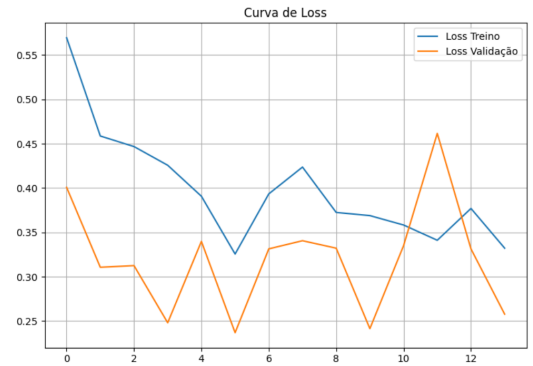

Figura 5.

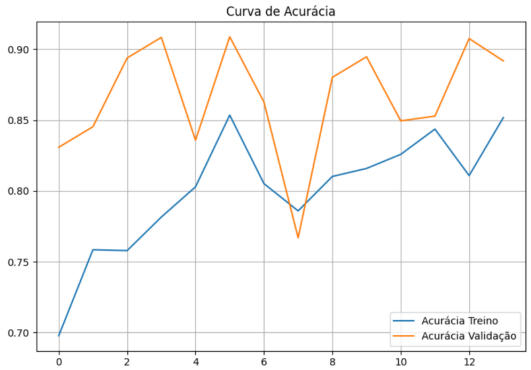

Figura 6.

3) Performance no Conjunto de Teste: **Métricas Principais:**

- **Acurácia Global:** 77% (1.890 de 2.440 predições corretas)
- **Recall de Falhas:** 78.6% (528 de 672 falhas detectadas)
- **Especificidade:** 77.0% (1.362 de 1.768 casos normais corretos)
- **Precision de Falhas:** 57% (528 corretos em 934 predições de falha)
- **F1-Score de Falhas:** 0.658
- **AUC-ROC:** 0.854
- **Falhas Perdidas:** 144/672 (21.4%)

A Figura 7 (Distribuição de Probabilidades) revela separação clara mas incompleta entre as classes: casos normais concentram-se fortemente em probabilidades baixas (<0.2), enquanto falhas distribuem-se predominantemente em probabilidades altas (>0.8), com sobreposição significativa na região 0.5-0.7 explicando os erros de classificação. O threshold otimizado de 0.55 (representado pela linha verde) posiciona-se estrategicamente nesta região de transição.

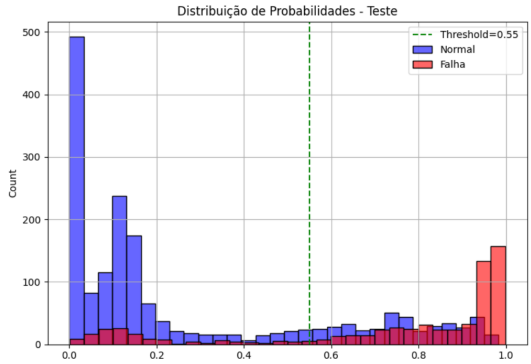

Figura 7.

A Figura 8 (Matriz de Confusão - Teste) quantifica os erros: 1.362 verdadeiros negativos (operação normal corretamente identificada), 406 falsos positivos (alarmes falsos - 23% dos casos normais), 144 falsos negativos críticos (falhas não detectadas - 21.4% das falhas reais), e 528 verdadeiros positivos (detecções corretas de falha).

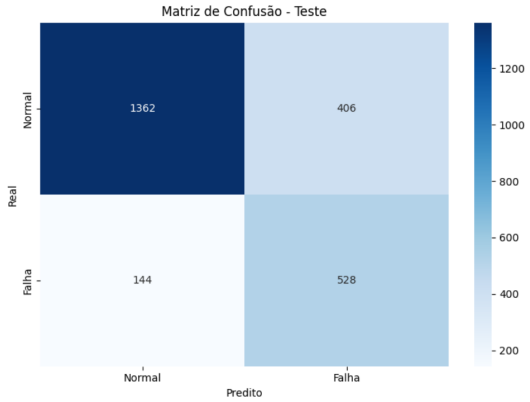

Figura 8.

4) Análise Crítica: O modelo demonstra capacidade sólida de detecção (recall 78.6%) mas apresenta grande quantidade de alarmes falsos (precision 57%). A taxa de 43% de falsos positivos pode comprometer a credibilidade operacional do sistema. Os 21.4% de falhas perdidas representam risco operacional que requer estratégias complementares de mitigação.

A queda de performance entre validação (F1=0.84, recall=0.87) e teste (F1=0.66, recall=0.79) sugere leve overfitting, embora o AUC-ROC de 0.854 confirme a capacidade discriminativa fundamental do modelo.

B. LSTM Classificação Binária - Threshold 90%

1) Processo de Otimização: Optuna convergiu no trial 17 com F1-score de validação de 0.8934, representando configuração significativamente diferente do modelo 80%: janela temporal 52% menor (26 vs. 54), arquitetura 31% mais compacta (39.837 vs. 57.457 parâmetros), e dropout 51% menor (0.20 vs. 0.41).

2) Resultados de Treinamento: O modelo treinou por 26 épocas (early stopping na época 18, restaurando pesos da época 11). A Figura 9 (Curva de Loss) mostra descida suave e monotônica da loss de treino de 0.52 para 0.12, com validação estabilizando em 0.27. A Figura 10 (Curva de Acurácia) apresenta crescimento de 71% para 95% no treino, com validação mantendo-se estável entre 81-88%.

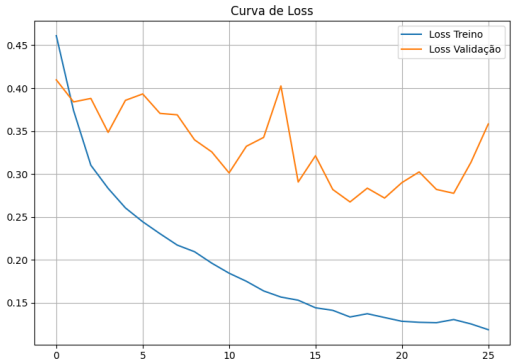

Figura 9.

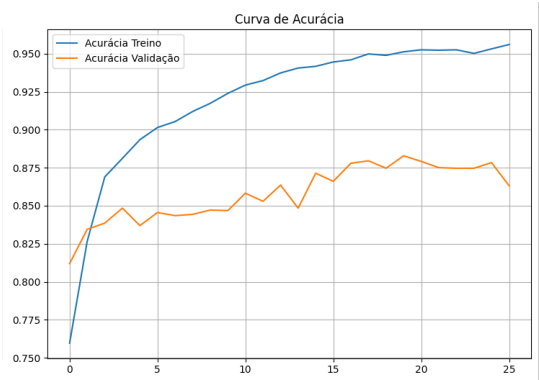

Figura 10.

3) Performance no Conjunto de Teste: **Métricas Principais:**

- **Acurácia Global:** 89% (2.203 de 2.464 predições corretas) - 12 p.p. superior ao modelo 80%
- **Recall de Falhas:** 82.6% (907 de 1.098 falhas detectadas)
- **Especificidade:** 94.9% (1.297 de 1.366 casos normais corretos)
- **Precision de Falhas:** 93% (907 corretos em 977 predições de falha)
- **F1-Score de Falhas:** 0.875 - 33% superior ao modelo 80%
- **AUC-ROC:** 0.969 - excelência discriminativa
- **Falhas Perdidas:** 191/1.098 (17.4%)

A Figura 11 (Distribuição de Probabilidades) mostra separação muito mais pronunciada entre classes comparada ao modelo 80%: casos normais concentram-se massivamente em probabilidades <0.1, enquanto falhas distribuem-se em picos claros próximos a 1.0. O threshold otimizado de 0.45 (abaixo do padrão 0.5) prioriza detecção de falhas mantendo precisão excepcional.

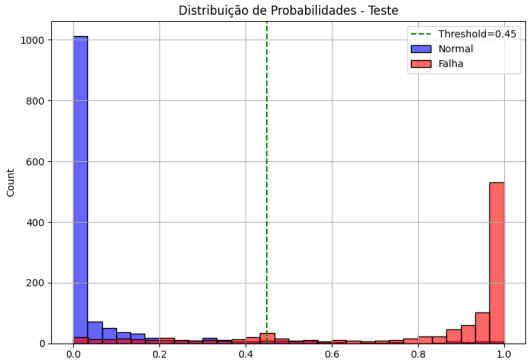

Figura 11.

A Figura 12 (Matriz de Confusão - Teste) revela performance balanceada superior: 1.297 verdadeiros negativos, apenas 69 falsos positivos (5% dos casos normais - redução de 83% vs. modelo 80%), 191 falsos negativos (17.4% das falhas), e 907 verdadeiros positivos.

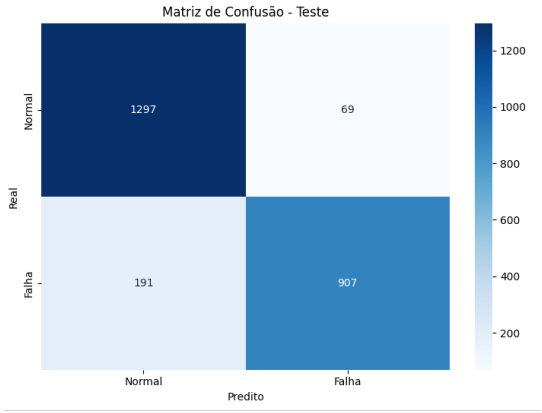

Figura 12.

4) Superioridade Estatística: O modelo 90% supera o modelo 80% em todas as métricas:

- Acurácia: +12 pontos percentuais (89% vs. 77%)
- Precision: +36 pontos percentuais (93% vs. 57%)
- Recall: +4 pontos percentuais (82.6% vs. 78.6%)
- F1-Score: +0.217 pontos (0.875 vs. 0.658)
- AUC-ROC: +0.115 pontos (0.969 vs. 0.854)
- Redução de falsos positivos: -82.9% (69 vs. 406)
- Redução de falhas perdidas: -4 pontos percentuais

Esta superioridade multidimensional é atribuída a três fatores: (1) balanceamento natural superior (44/56 vs. 27/73), facilitando aprendizado; (2) padrões de degradação inicial mais consistentes e detectáveis que falhas severas estabelecidas; (3) arquitetura mais enxuta reduzindo overfitting. Apesar disso, a janela temporal do modelo 90% é menor, 26 horas enquanto o modelo 80% possui 54 horas, o que pode ser um problema para a manutenção preventiva.

C. Modelos Baseados em Árvore - Classificação Multiclasse

1) Configuração do Problema: Os modelos Decision Tree e Random Forest abordaram classificação em três categorias com forte desbalanceamento: ≤60% (2.8%), 60-80% (24.1%), ≥80% (73.1%). A divisão estratificada manteve estas proporções em treino (10.709 amostras) e teste (5.789 amostras).

2) Decision Tree - Resultados: O GridSearchCV explorou 480 configurações em 3-fold cross-validation (1.440 ajustes totais), selecionando: criterion=entropy, max_depth=10, max_features=sqrt, min_samples_split=10, min_samples_leaf=1, com F1-score médio de CV de 0.7400.

Performance no Teste:

- **Acurácia Global:** 78%
- **Classe ≤60% (Crítica):** Precision=0.55, Recall=0.70, F1=0.62
- **Classe 60-80%:** Precision=0.42, Recall=0.44, F1=0.43
- **Classe ≥80%:** Precision=0.91, Recall=0.82, F1=0.86
- **F1-Score Ponderado:** 0.7914

A Figura 13 (Matriz de Confusão DT - Teste) revela performance desigual: a classe majoritária (≥80%) alcança 3.480 acertos de 4.228 (82%), enquanto as classes minoritárias apenas 73 de 166 (44%) corretos em 60-80%, e 980 de 1.395 (70%) em ≤60%. Apresenta confusão entre classes adjacentes.

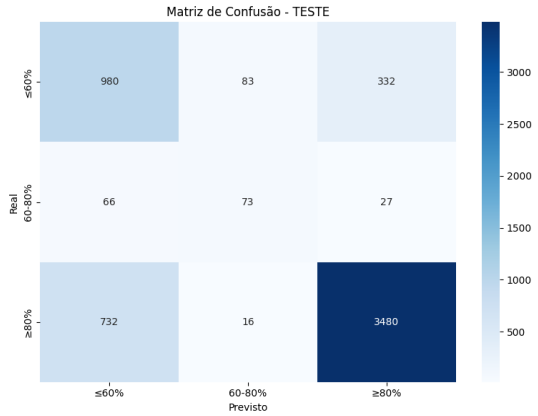

Figura 13.

A diferença treino-teste (F1-score: 0.9218 vs. 0.7914, gap de 0.1304) indica overfitting moderado, esperado em árvores de decisão únicas sem ensemble.

3) Random Forest - Resultados: O RandomizedSearchCV testou 50 configurações aleatórias (150 ajustes totais), convergindo para: n_estimators=300, max_depth=None, criterion=gini, max_features=sqrt, min_samples_split=2, min_samples_leaf=2, bootstrap=True, com F1-score médio de CV de 0.7405.

Performance no Teste:

- **Acurácia Global:** 87% (+9 p.p. vs. Decision Tree)
- **Classe ≤60% (Crítica):** Precision=0.73, Recall=0.76, F1=0.74 (+0.12 vs. DT)
- **Classe 60-80%:** Precision=0.63, Recall=0.39, F1=0.48 (+0.05 vs. DT)
- **Classe ≥80%:** Precision=0.92, Recall=0.92, F1=0.92 (+0.06 vs. DT)
- **F1-Score Ponderado:** 0.8670 (+7.6 p.p. vs. DT)
- **F1-Score Macro:** 0.714

A Figura 14 (Matriz de Confusão RF - Teste) mostra melhoria substancial: classe ≤60% alcança 1.058 de 1.395 corretos (75.8%, +5.8 p.p.), classe 60-80% mantém desafio com 64 de 166 (38.6%), e classe ≥80% atinge excelência com 3.906 de 4.228 (92.4%, +10.4 p.p.).

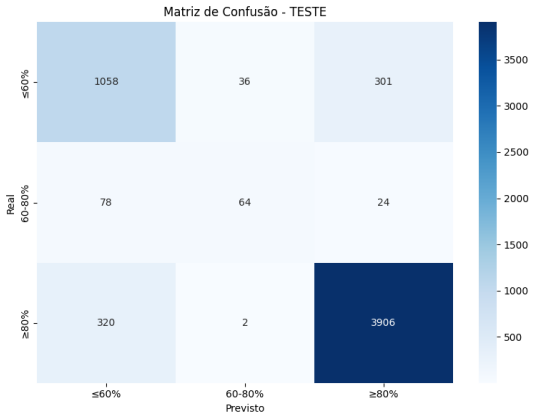

Figura 14.

Figure 15 (Feature Importance) reveals that heat_1001_12 (16%) leads, followed by current_eletrica_1001A (14.5%), temperature_1001_26 (13%), flow_1001_15 (12.5%), flow_1001_7 (10.5%), current_1001B (9.5%), pressure_1001_10 (6.5%), input_analogica (6%), temperature_1001 (6%), and temperature_9401 (4.5%). Esta hierarquia indica que variáveis térmicas e elétricas são os principais indicadores de estado da válvula.

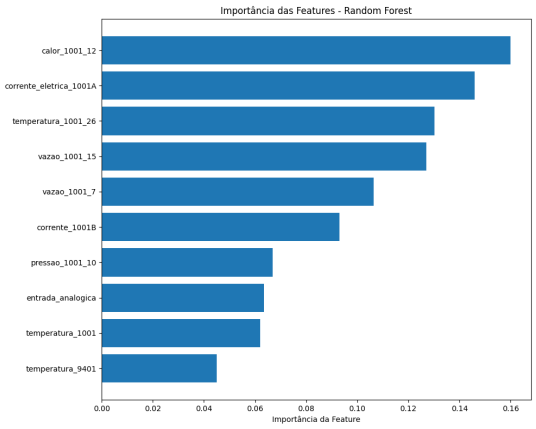

Figura 15.

A Figura 16 (Métricas por Classe) visualiza o desempenho balanceado entre precision, recall e F1-score para cada categoria, destacando a excelência na classe ≥80% (scores >0.9) e o desafio persistente na classe intermediária 60-80% (scores ∼0.4-0.6).

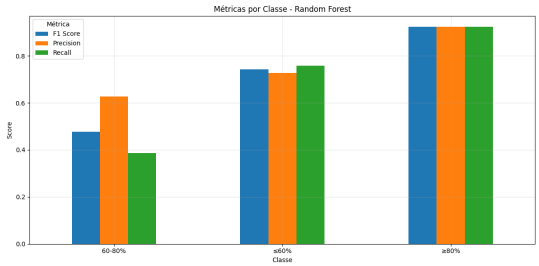

Figura 16.

O gap treino-teste (F1-score: 0.9963 vs. 0.8670, diferença de 0.1292) é similar ao Decision Tree, mas o ensemble de 300 árvores proporciona estabilidade superior apesar do ajuste perfeito no treino.

4) Análise da Classe Crítica ≤60%: O Random Forest detectou 1.058 de 1.395 casos críticos (recall 75.8%), perdendo 337 casos (24.2%). Dos 1.456 alertas emitidos para esta classe, 398 foram falsos positivos (precision 72.7%). Em contexto industrial, este trade-off é aceitável, cada 10 alertas contêm 7 casos reais, e 3 de cada 4 situações críticas são detectadas.

D. Discussão e Recomendações

1) Comparação Quantitativa Entre Todos os Modelos: A Tabela 1 consolida as métricas principais dos quatro modelos avaliados no conjunto de teste:

Tabela I
COMPARAÇÃO DE PERFORMANCE NO CONJUNTO DE TESTE

Métrica	**LSTM 80**	**LSTM 90**	**DT**	**RF**
Acurácia (%)	77	**89**	78	87
F1 Pond.	0.78	**0.89**	0.79	0.87
F1 Crítico	0.66	**0.87**	0.62	0.74
Recall Crít. (%)	78.6	82.6	70	**75.8**
Precis. Crít. (%)	57	**93**	55	73
AUC-ROC	0.85	**0.97**	–	–
Falhas Perd. (%)	21.4	**17.4**	30*	24.2*
Alarms Fals. (%)	43	**7**	45*	27*
Parâm.	57k	**40k**	–	300
Window	54	**26**	–	–

*Valores ref. classe ≤60%

2) Modelo Vencedor: O **LSTM Classificação 90%** apresenta solução superior, liderando em 8 de 10 métricas avaliadas. Suas vantagens decisivas incluem:

- **Precision (93%):** Elimina fadiga de alertas, crítico para aceitação operacional
- **Balanço otimizado:** Recall competitivo (82.6%) com mínimos falsos positivos (7%)
- **Eficiência computacional:** Menor janela (26) e menos parâmetros (39.837) viabilizam sua execução
- **Robustez discriminativa:** AUC-ROC 0.969 indica separação quase perfeita das classes

3) Aplicação dos Demais Modelos: Embora o LSTM 90% seja superior, os outros modelos têm papéis complementares:

LSTM 80%: Útil como segundo nível de alerta para confirmar criticidade extrema. Recall de 78.6% garante que falhas severas não passam despercebidas mesmo com falsos positivos do modelo 90%.

Random Forest: Oferece interpretabilidade através da importância de features (Figura 11), essencial para entendimento operacional. Adequado para análises offline e identificação de causas raiz.

Decision Tree: Educacional para visualização de regras de decisão, mas desempenho inferior inviabiliza uso em produção.

4) Arquitetura Recomendada para Produção: Propõe-se sistema hierárquico de três camadas:

Camada 1 - Monitoramento Contínuo: LSTM 90% operando em tempo real (inferência a cada minuto), gerando alertas de *atenção* quando probabilidade > 0.45.

Camada 2 - Confirmação de Criticidade: Quando LSTM 90% emite alerta, é acionado o LSTM 80% sobre janela estendida (54 timesteps). Se ambos concordam (válvula $<$ 90% & $<$ 80%), cria-se um alerta *crítico*.

Camada 3 - Análise de Causa Raiz: Random Forest executa classificação multiclasse e gera relatório de importância de features específico para o evento, auxiliando no diagnóstico.

Esta arquitetura maximiza sensibilidade (camada 1), minimiza falsos positivos (camada 2), e fornece insights acionáveis (camada 3), alinhando-se com requisitos industriais de confiabilidade, explicabilidade e tempo de resposta.

IV. CONCLUSÃO

Portanto o modelo LSTM-Classificação com threshold de 90% (LSTM-90) demonstrou o melhor desempenho, com acurácia de 89%. O Random Forest para classificação multiclasse também apresentou resultados satisfatórios, com acurácia de 87%.

Os experimentos realizados revelam comportamentos distintos entre as abordagens de classificação e regressão para predição do estado da válvula. Testes com modelos de LSTM-Regressão apresentaram overfitting crítico. Com a validação cruzada temporal evidenciando alta variância ($R^2 = 0.43 \pm 0.42$) e drift temporal significativo (Fold 5: $R^2 = -0.33$), é possível concluir que o número de casos de falha aumentaram nos últimos mesês, o que dificultou a previsão da abertura da válvula. Dessa forma, LSTM-Regressão tendeu a prever valores maiores do que os reais no teste, falhando em capturar quedas abruptas que se tornaram mais frequentes.

REFERÊNCIAS

[1] ESTATÍSTICA FÁCIL, "O que é Kernel Density Estimation (KDE)," [Online]. Disponível em: https://estatisticafacil.org/glossario/o-que-e-kernel-density-estimation/. Acesso em: 10 jul. 2025.

[2] GEEKSFORGEEKS, "Data Visualization with Pairplot – Seaborn and Pandas," [Online]. Disponível em: https://www.geeksforgeeks.org/data-visualization-with-pairplot-seaborn-and-pandas/. Acesso em: 10 jul. 2025.

[3] P. SILVA, "Como visualizar e analisar dados com Python," *Turing Talks – Medium*, 2020. [Online]. Disponível em: https://medium.com/turing-talks/como-visualizar-e-analisar-dados-com-python-f209bfbae68e. Acesso em: 17 ago. 2025.

[4] M. RIBEIRO, "Introdução às redes LSTM: prevendo valor de ações na bolsa," *DataRisk IO – Medium*, 2021. [Online]. Disponível em: https://medium.com/datarisk-io/introdu%C3%A7%C3%A3o-%C3%A0s-redes-lstm-prevendo-valor-de-a%C3%A7%C3%B5es-na-bolsa-df270ca0cee5. Acesso em: 17 ago. 2025.

[5] H. KABHI, “Understanding LSTM for Sequence Classification: A Practical Guide with PyTorch,” *Medium*, 2021. [Online]. Disponível em: https://medium.com/@hkabhi916/understanding-lstm-for-sequence-classification-a-practical-guide-with-pytorch-ac40e84ad3d5. Acesso em: 13 set. 2025.

[6] GEEKSFORGEEKS, “Hyperparameter Tuning with Optuna in PyTorch,” [Online]. Disponível em: https://www.geeksforgeeks.org/deep-learning/hyperparameter-tuning-with-optuna-in-pytorch/. Acesso em: 19 set. 2025.

[7] Y. ZHENG, C. YANG e T. NAKAGAWA, “Industrial predictive maintenance using machine learning: a review,” *Journal of Intelligent Manufacturing*, vol. 33, no. 2, pp. 435–458, 2022. DOI: 10.1007/s10845-021-01842-4.

[8] L. SAIDI, H. B. ABDALLAH e B. CHEBEL-MORELLO, “A comparison of Random Forest and Decision Tree algorithms for failure prediction,” *Procedia Computer Science*, vol. 175, pp. 235–240, 2020. DOI: 10.1016/j.procs.2020.07.037.

[9] C. A. GARCIA *et al.*, “Predictive maintenance in industrial systems: Combining LSTM networks and decision tree classifiers,” *Applied Sciences*, vol. 10, no. 20, p. 7210, 2020. DOI: 10.3390/app10207210.